

Model-based clustering and classification

part 3: network data and co-clustering

Julien JACQUES

Université Lumière Lyon 2

Mixture model for network data

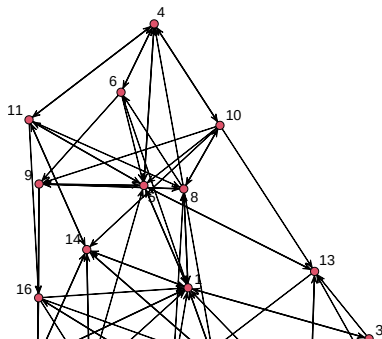
Co-clustering

Mixture model for network data

Network data

Network data arise when the relational data are collected on pairs of entities

```
library(network)
data(sampson , package="ergm")
xSampson <- as.matrix(samplike)
layout <- network.layout.fruchtermanreingold(samplike,
                                             layout.par=NULL)
plot.network(samplike, label=1:18, mode="fruchtermanreingold",
             coord=layout)
```



Network data

Notations

- ▶ n : number of individuals, **nodes** in the graph
- ▶ x_{ij} relationship between nodes i and j
 - ▶ if $x_{ij} = 1$ if node i and j are linked, 0 otherwise, we talk about **binary** graph. But
 - ▶ but x_{ij} can be also in $\mathbb{N}, \mathbb{R}..$
- ▶ we consider **undirected** graph: $x_{ij} = x_{ji}$

Network data

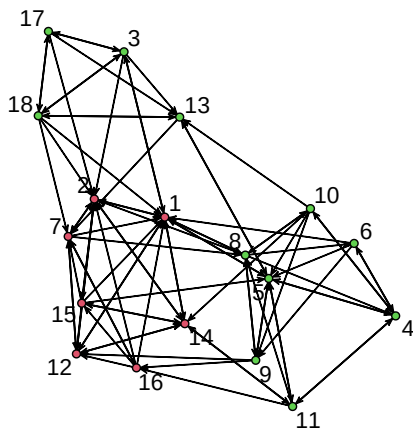
Example: Sampson's Monk data recorder interaction between 18 novice monks in a monastery.

```
head(xSampson)
```

```
##           John Bosco Gregory Basil Peter Bonaventure Berthold
## John Bosco           0         1     1     0           1
## Gregory              1         0     0     0           0
## Basil                1         1     0     0           0
## Peter                0         0     0     0           1
## Bonaventure          1         0     0     1           0
## Berthold             1         0     0     1           1
##           Ambrose Romauld Louis Winfrid Amand Hugh Boniface
## John Bosco           0         0     0         1     0     1     0
## Gregory              0         0     0         1     0     1     1
## Basil                0         0     0         0     1     0     0
## Peter                0         1     1         0     0     0     0
## Bonaventure          1         0     1         0     1     0     0
## Berthold             1         0     0         0     0     0     0
##           Simplicius
## John Bosco           0
```

Network clustering

The goal is to detect **communities** in the network: subsets of nodes which have a tendency to form link with each other



The Stochastic Block Model

The Stochastic Block Model (SBM) assumes:

- ▶ there is G communities/blocks in the network
- ▶ a node come from block g with probability p_g
- ▶ α_{gh} : probability that a node in block g is related to a node in block h
- ▶ $\tilde{z}_i = (\tilde{z}_{i1}, \dots, \tilde{z}_{iG})$: membership of node i to the blocks
- ▶ all pairwise interactions are independent

Consequently:

$$p(x_{ij} = 1 | \alpha, \tilde{z}_{ig} = 1, \tilde{z}_{jh} = 1) = \alpha_{gh} = \tilde{\mathbf{z}}_i' \boldsymbol{\alpha} \tilde{\mathbf{z}}_j$$

with $\boldsymbol{\alpha}$ the $G \times G$ -matrix of α_{gh}

Snijders, T. A. B., and Nowicki, K. 1997. Estimation and prediction for stochastic blockmodels for graphs with latent block structure. Journal of Classification, 14(1), 75–100. 298, 300

SBM inference

We are in face to a likelihood maximization according to parameter $\theta = (\mathbf{p}, \alpha)$ with missing variables $\tilde{\mathbf{z}}$. The log-likelihood:

$$L(\theta, \mathbf{x}) = \sum_{\tilde{\mathbf{z}}} p(\mathbf{x}, \tilde{\mathbf{z}}; \theta)$$

involves the summation over the G^n possible partitions $\tilde{\mathbf{z}}$ of the n nodes into G groups, which is intractable.

SBM inference

The EM algorithm is well adapted for likelihood maximization in presence of missing variables $\tilde{\mathbf{z}}$.

The complete log-likelihood is:

$$L_c(\boldsymbol{\theta}, \mathbf{x}, \tilde{\mathbf{z}}) = \prod_{i=1}^n \prod_{g=1}^G p_g^{\tilde{z}_{ig}} \times \prod_{i=1}^{n-1} \prod_{j=i+1}^n (\tilde{\mathbf{z}}_i \boldsymbol{\alpha} \tilde{\mathbf{z}}_j)^{x_{ij}} (1 - \tilde{\mathbf{z}}_i \boldsymbol{\alpha} \tilde{\mathbf{z}}_j)^{1-x_{ij}}$$

and its logarithm

$$\ell_c(\boldsymbol{\theta}, \mathbf{x}, \tilde{\mathbf{z}}) = \sum_{i=1}^n \sum_{g=1}^G \tilde{z}_{ig} \ln p_g + \underbrace{\sum_{i=1}^{n-1} \sum_{j=i+1}^n x_{ij} \ln(\tilde{\mathbf{z}}_i \boldsymbol{\alpha} \tilde{\mathbf{z}}_j) + (1 - x_{ij}) \ln(1 - \tilde{\mathbf{z}}_i \boldsymbol{\alpha} \tilde{\mathbf{z}}_j)}_A$$

where

$$A = \sum_{i=1}^{n-1} \sum_{g=1}^G \sum_{j=i+1}^n \sum_{h=1}^G x_{ij} \tilde{z}_{ig} \tilde{z}_{jh} \ln(\alpha_{gh}) + (1 - x_{ij}) \ln(1 - \alpha_{gh}) \tilde{z}_{ig} \tilde{z}_{jh}$$

The number of terms in A is $n(n-1)G^2$.

SBM inference

The E-step of the EM algorithm for the SBM model leads to compute the $n(n-1)G^2$ terms:

$$E_{\theta^{(q)}}[\tilde{z}_{ig}\tilde{z}_{jh}|x_{ij}] = p_{\theta^{(q)}}(\tilde{z}_{ig}\tilde{z}_{jh} = 1|x_{ij})$$

which are not factorizable: $\tilde{z}_i$ and $\tilde{z}_j$ are not independent since it depends on x_{ij} , the fact there is or not a link between nodes i and j .

Consequently, the E step is generally intractable.

We should use alternatives: Gibbs sampling or Variational approximation.

Variational approximation

Since $p_{\theta}(\mathbf{z}|\mathbf{x})$ can not be calculated, we need to approximate by any $q(\mathbf{z})$.

For any $q(\mathbf{z})$, we have⁽¹⁾:

$$\begin{aligned}\ln p_{\theta}(\mathbf{x}) &\geq \ln p_{\theta}(\mathbf{x}) - KL(p_{\theta}(\tilde{\mathbf{z}}|\mathbf{x}), q(\tilde{\mathbf{z}})) \\ &= E_q[\ln p_{\theta}(\mathbf{x}, \tilde{\mathbf{z}})] - E_q[\ln q(\tilde{\mathbf{z}})] \\ &= \mathcal{J}_{\theta}(q)\end{aligned}\tag{1}$$

Equality in (1) occur for $q(\tilde{\mathbf{z}}) = p_{\theta}(\tilde{\mathbf{z}}|\mathbf{x})$

⁽¹⁾ see for instance p29 of

<http://www-math.univ-poitiers.fr/~yslaoui/2019-EL-HAJ-Abir-These.pdf>

Variational EM algorithm

The principle of the Variational EM (VEM) algorithm is to maximize the lower bound $\mathcal{J}_\theta(q)$

Since equality occurs when $q(\tilde{\mathbf{z}}) = p_\theta(\tilde{\mathbf{z}}|\mathbf{x})$, the VEM consists in alternating 2 steps:

- ▶ VE step: find the best lower bound $\mathcal{J}_\theta(q)$ according to q .
This is equivalent to find $q^* = \operatorname{argmin}_q KL(p_\theta(\tilde{\mathbf{z}}|\mathbf{x}), q(\tilde{\mathbf{z}}))$
- ▶ M step: as usually, find

$$\theta^* = \operatorname{argmax}_\theta E_{q^*}[\ln p_\theta(\mathbf{x}, \tilde{\mathbf{z}})] \quad (2)$$

All the interest is to restrict $q(\mathbf{z})$ to a family of functions which makes (2) tractable.

Mean-field approximation

Mean-field approximation consider factorizable $q(\tilde{\mathbf{z}})$:

$$q_{\tau}(\mathbf{z}) = \prod_i q(\tilde{z}_i, \tau_i) = \prod_i \prod_g \tau_{ig}^{\tilde{z}_{ig}}$$

Variational EM algorithm

Starting from initialization of variational parameter $\tau^{(0)}$, alternates:

- ▶ *pseudo- E step*: update τ :

$$\tau^{(t+1)} = \operatorname{argmax}_{\tau} \mathcal{J}_{\theta^{(t)}}(q_{\tau})$$

- ▶ *M step*: update θ :

$$\theta^{(t+1)} = \operatorname{argmax}_{\theta} \mathcal{J}_{\theta}(q_{\tau^{(t+1)}})$$

until $\|\theta^{(t+1)} - \theta^{(t)}\| < \epsilon$

Node clustering

Once the VEM algorithm has converged, the node clustering can be obtained from the variational distribution of $\mathbf{z}$: node i is affected to cluster g which maximizes $\hat{\tau}_{ig}$

Model selection

We have to choose the number of communities.

For the same reason that the EM is not tractable, the likelihood and then the BIC criterion are not tractable.

But the ICL criterion, based on the complete likelihood is tractable, by approximating $\underline{z}$ by $\hat{\underline{z}}$.

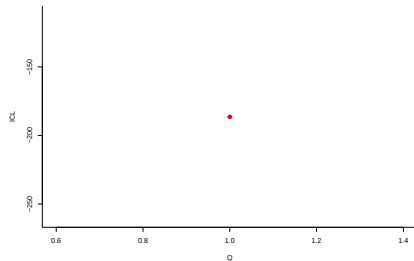
$$ICL = \ell(\hat{\boldsymbol{\theta}}, \underline{\mathbf{x}}, \hat{\underline{\mathbf{z}}}) - \frac{\nu}{2} \ln 2\pi$$

where $\nu = G - 1 + G(G + 1)/2$ is the total number of SBM parameters.

SBM in R

```
library(blockmodels)
my_model <- BM_bernoulli("SBM", xSampson)
my_model$estimate()
```

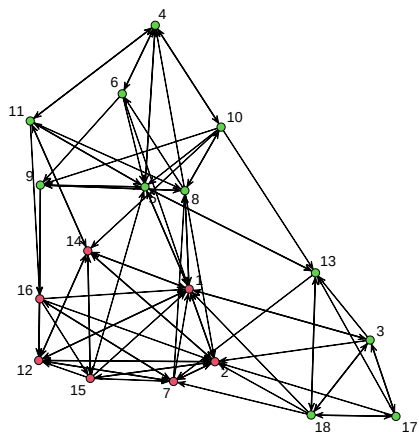
```
## -> Estimation for 1 groups
##           -> 1 initializations provided
##           -> 0 initializations already used
##           -> Estimation with 1 initializations
## Executing 1 jobs in parallel
##           -> new ICL: -186.453270386301
##           -> old ICL: NA
```



```
## -> Computation of eigen decomposition used for initializations
```

SBM in R

```
G=which.max(my_model$ICL)
z <- my_model$memberships[[G]]
plot.network(samplike, label=1:18, mode="fruchtermanreingold",
            coord=layout, vertex.col=max.col(z$Z)+1)
```



Exercise

The **PoliticalBlogs** data set shows the linking structure in online blogs which commentate on French political issues; the data were collected by Observatoire Presidentielle in October 2006

Let use SBM to detect communities in the PoliticalBlogs network

```
library(MBCbook)  
data(PoliticalBlogs)
```

Exercise

First we have to transform the network format into matrix

```
library(network)
xPoliticalBlogs=as.matrix(PoliticalBlogs)
```

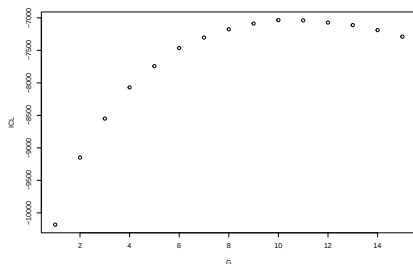
Then we can estimate the SBM model:

```
my_model <- BM_bernoulli("SBM",xPoliticalBlogs,plotting="")
my_model$estimate()
```

Exercise

ICL allows to choose the number of communities:

```
plot(1:length(my_model$ICL),my_model$ICL,ylab="ICL",xlab='G')
```



and the best one is given by:

```
which.max(my_model$ICL)
```

```
## [1] 10
```

Exercise

We can plot the obtained communities

```
G=which.max(my_model$ICL)
z <- my_model$memberships[[G]]
plot.network(PoliticalBlogs, mode="fruchtermanreingold",
  vertex.col=max.col(z$Z))
```



Exercise

We can compare the obtained communities with the political parties

```
partis=NULL
for (i in 1:196){
  partis=c(partis,PoliticalBlogs$val[[i]]$group)
}
classes=apply(z$Z,1,which.max)
table(classes,partis)
```

##		partis							
##	classes	Analysts	Lib	Other	PRG	PS	UDF	UMP	
##	1	0	24	0	0	0	0	0	
##	2	1	0	0	0	0	24	2	
##	3	0	0	0	0	0	1	24	
##	4	0	0	2	1	26	0	0	
##	5	0	0	0	0	5	0	0	
##	6	2	1	18	9	6	0	1	
##	7	0	0	0	0	0	0	11	
##	8	1	0	0	0	0	6	0	
##	9	7	0	0	1	2	1	2	
##	10	0	0	0	0	18	0	0	

Exercise

We can have a look to the model parameters which give the probability connexion between groups

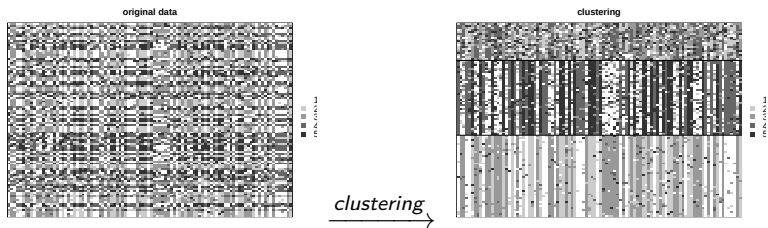
```
my_model$model_parameters[[10]]
```

```
## $pi
##           [,1]           [,2]           [,3]           [,4]           [,5]
## [1,] 0.5430323586 0.0020304638 0.008794024 0.0033935529 0.03489550
## [2,] 0.0020304638 0.2124711898 0.004841295 0.0004642879 0.01637355
## [3,] 0.0087940245 0.0048412945 0.343603254 0.0034137785 0.04882274
## [4,] 0.0033935529 0.0004642879 0.003413778 0.2405446298 0.49860225
## [5,] 0.0348954968 0.0163735461 0.048822739 0.4986022505 0.86858830
## [6,] 0.0036954444 0.0022575804 0.005439865 0.0243461562 0.06921560
## [7,] 0.0008982470 0.0073510535 0.036606262 0.0039424885 0.02082469
## [8,] 0.0076431204 0.6147688792 0.085641424 0.0892882586 0.22744936
## [9,] 0.0926465908 0.0345385248 0.107152427 0.0905327753 0.42207750
## [10,] 0.0008935681 0.0007351848 0.002995476 0.1392273317 0.82739925
##           [,6]           [,7]           [,8]           [,9]           [,10]
## [1,] 0.0036954444 0.0008982470 0.007643120 0.09264659 0.0008935681
## [2,] 0.0022575804 0.0073510535 0.614768879 0.03453852 0.0007351848
## [3,] 0.0054398652 0.0366062616 0.085641424 0.10715243 0.0029954756
## [4,] 0.0243461562 0.0039424885 0.089288259 0.09053278 0.1392273317
## [5,] 0.0692156019 0.0208246931 0.227449358 0.42207750 0.8273992466
## [6,] 0.0655818226 0.0004001878 0.053378880 0.07629503 0.0178451302
## [7,] 0.0004001878 0.9102002170 0.002446394 0.04260576 0.0013670588
## [8,] 0.0533788796 0.0024463940 0.973641330 0.37871052 0.0260534789
## [9,] 0.0762950345 0.0426057557 0.378710518 0.37884601 0.0565520466
```

Co-clustering

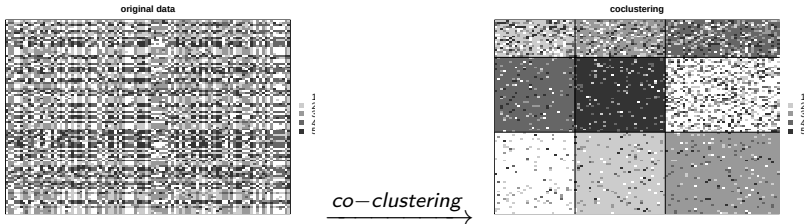
Co-clustering

When the number of features becomes large. . . cluster interpretation is impossible



Co-clustering

Co-clustering clusterizes both the observations (row) and the features (columns)



Co-clustering notations

- ▶ K : number of row clusters
- ▶ L : number of column clusters
- ▶ row-cluster partitions: $\mathbf{v} = (v_{ik})_{i,k}$,
s.t. $v_{ik} = 1$ if row i belong to row-cluster k
- ▶ column-cluster partitions: $\mathbf{w} = (w_{h\ell})_{h,\ell}$,
s.t. $w_{h\ell} = 1$ if column h belong to row-cluster ℓ
- ▶ α_k : row-cluster propotion
- ▶ β_ℓ : column-cluster propotion
- ▶ $\pi_{k\ell}$: the parameter of the distribution in block $k\ell$ (the distribution depends on the nature of the data)
- ▶ $\theta = (\pi_{k\ell}, \alpha_k, \beta_\ell)$: the whole set of LBM parameter

Latent Block Model (LBM)

LBM Assumption 1:

- ▶ $\mathbf{v}$ and $\mathbf{w}$ are independent

Consequently, LBM likelihood is

$$p(\mathbf{x}; \boldsymbol{\theta}) = \sum_{\mathbf{v} \in V} \sum_{\mathbf{w} \in W} p(\mathbf{v}; \boldsymbol{\theta}) p(\mathbf{w}; \boldsymbol{\theta}) p(\mathbf{x} | \mathbf{v}, \mathbf{w}; \boldsymbol{\theta})$$

with

- ▶ V (W) set of possible partitions of rows (columns) into K (L) groups,
- ▶ $p(\mathbf{v}; \boldsymbol{\theta}) = \prod_{ik} \alpha_k^{v_{ik}}$ and $p(\mathbf{w}; \boldsymbol{\theta}) = \prod_{h\ell} \beta_\ell^{w_{h\ell}}$

Latent Block Model (LBM)

LBM Assumption 2:

- ▶ once the row $\mathbf{v}$ and column $\mathbf{w}$ partitions are fixed, the $n \times p$ random variables $\mathbf{x}$ are assumed to be independent, identitically distributed in each block by $p(\cdot; \pi_{k\ell})$

$$p(\mathbf{x}; \theta) = \sum_{\mathbf{v} \in V} \sum_{\mathbf{w} \in W} \prod_{ik} \alpha_k^{v_{ik}} \prod_{h\ell} \beta_\ell^{w_{h\ell}} \prod_{ihk\ell} p(x_{ih}; \pi_{k\ell})^{v_{ik} w_{h\ell}}$$

LBM parcimony

Summarizing the $n \times p$ data matrix with univariate distributions in the $K \times L$ groups is very parsimonious, in comparison to a clustering in which the distribution in the K clusters would be p -variate.

Even if we are not interested in co-clustering, but only in clustering, LBM model can be seen as a data-driven parsimonious clustering model

Example in the Gaussian case:

- ▶ with $n = p = 100$
- ▶ clustering only with GMM: with $K = 3$, the number of parameters is $(3 - 1) + 3 \times 100 + 3 \times 100 \times 101/2 = 15452$
- ▶ LBM co-clustering: with $K = L = 3$, the number of parameters is $(3 - 1) + (3 - 1) + 3 \times 3 \times (1 + 1) = 22$

LBM likelihood

$$p(\mathbf{x}; \theta) = \sum_{\mathbf{v} \in V} \sum_{\mathbf{w} \in W} p(\mathbf{v}; \theta) p(\mathbf{w}; \theta) p(\mathbf{x} | \mathbf{v}, \mathbf{w}; \theta)$$

The likelihood is not computationally tractable due to the double missing structure ($\mathbf{v}$ and $\mathbf{w}$):

- ▶ $\#V = K^n$,
- ▶ $\#W = L^p$
- ▶ $\Rightarrow \sum_{\mathbf{v}, \mathbf{w}}$ has $K^n L^p$ terms

LBM inference

The aim is to estimate θ by maximizing the observed log-likelihood.

Classical approach is to use an EM algorithm, which alternates

- ▶ E step: compute

$$E_{\theta^{(q)}}[\ell_c(\theta; \mathbf{x}, \mathbf{v}, \mathbf{w}) | \mathbf{x}] = E_{\theta^{(q)}}[\ln p(\mathbf{x}, \mathbf{v}, \mathbf{w}; \theta) | \mathbf{x}].$$

- ▶ M step: update θ by

$$\theta^{(q+1)} = \operatorname{argmax}_{\theta} E_{\theta^{(q)}}[\ln p(\mathbf{x}, \mathbf{v}, \mathbf{w}; \theta) | \mathbf{x}]$$

But for the same reason that the likelihood is not tractable, the **E step is also not tractable**.

Stochastic EM

Let remind that the **Stochastic EM** algorithm is a variant of the EM algorithm, obtained by *generating* the missing variable at the SE step:

- ▶ SE step:
 - ▶ generate missing data $(\mathbf{v}, \mathbf{w})$ cond. on $\mathbf{x}$ and $\boldsymbol{\theta}^{(q)}$
- ▶ M step:
 - ▶ update $\boldsymbol{\theta}^{(q)}$ in $\boldsymbol{\theta}^{(q+1)}$ maximizing the complete likelihood

After a given number of iterations, $\hat{\boldsymbol{\theta}}$ is obtained from the sampling distribution (after a burn-in period)

Stochastic EM within Gibbs

Since the distribution of $(\mathbf{v}, \mathbf{w})$ is not tractable, the idea is to generate them using a Gibbs sampler, which alternates on few iterations:

- ▶ SE-Gibbs step:
 - ▶ generate $\mathbf{v}$ cond. on $(\mathbf{x}, \mathbf{w})$ and $\theta^{(q)}$
 - ▶ generate $\mathbf{w}$ cond. on $(\mathbf{x}, \mathbf{v})$ and $\theta^{(q)}$
- ▶ M step:
 - ▶ update $\theta^{(q)}$ in $\theta^{(q+1)}$ maximizing the complete likelihood

After a given number of iterations, $\hat{\theta}$ is obtained from the sampling distribution (after a burn-in period)

Partition estimation $(\hat{\mathbf{v}}, \hat{\mathbf{w}})$ can be obtained with a Gibbs sampler using $\hat{\theta}$.

Model selection

Since the likelihood is not tractable, BIC is also not tractable.

We can use ICL which is based on the complete likelihood, by approximating it using $(\hat{\mathbf{v}}, \hat{\mathbf{w}})$:

$$ICL = \ln p(\mathbf{x}, \hat{\mathbf{v}}, \hat{\mathbf{w}}; \boldsymbol{\theta}) - \frac{K-1}{2} \log n - \frac{L-1}{2} \log d - \frac{KL\nu}{2} \log(nd)$$

where ν is the number of model parameter per block.

LBM in practice

```
library(blockcluster)
data(binarydata)
out<-coclusterBinary(binarydata,nbcoccluster=c(2,3))
```

Co-Clustering successfully terminated!

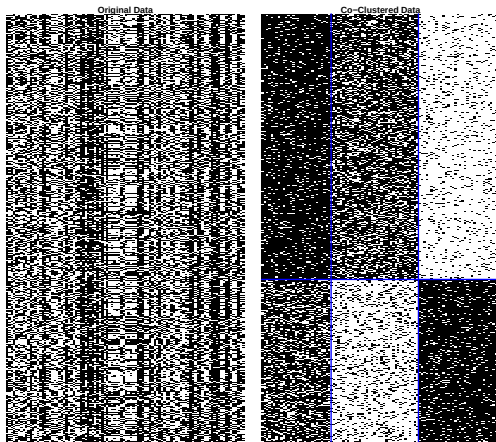
LBM in practice

```
summary(out)
```

```
## *****  
## Model Family : Bernoulli Latent block model  
## Model Name : pik_rhol_epsilonkl  
## Co-Clustering Type : Unsupervised  
## ICL value: -45557.07  
##  
## Model Parameters..  
##  
## Class Mean:  
##      [,1] [,2] [,3]  
## [1,] FALSE FALSE TRUE  
## [2,] FALSE TRUE FALSE  
##  
## Class Dispersion:  
##      [,1] [,2] [,3]  
## [1,] 0.09798014 0.3022391 0.1011803  
## [2,] 0.30176927 0.2003679 0.1006314  
##  
## Row proportions: 0.618 0.382
```

LBM in practice

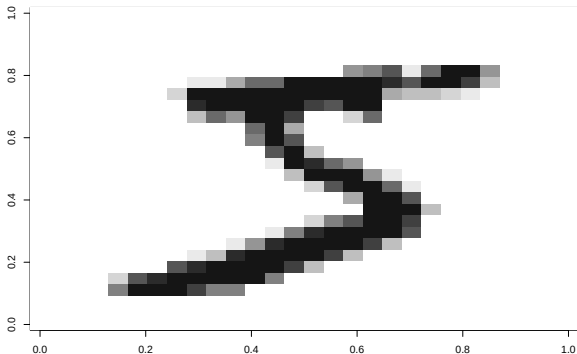
```
plot(out)
```



Exercise 1

Sur une des images des données MNIST, réaliser un co-clustering de l'image avec le package `blockcluster`.

```
load("MNIST.Rdata")
show_digit <- function(x, col=gray(12:1/12)){
  image(matrix(x, nrow=28)[,28:1], col=col)
}
show_digit(train$x[1,])
```



Exercise 2

The goal is to provide a analysis of the Young People Survey available at <https://www.kaggle.com/cardot/se-young-people-survey/data> with the `ordinalClust` package.